

**FUNDAÇÃO EDUCACIONAL MIGUEL MOFARREJ
CENTRO UNIVERSITÁRIO DAS FACULDADES
INTEGRADAS DE OURINHOS
ENGENHARIA DE SOFTWARE**

**SGE: SISTEMA DE GERENCIAMENTO DE EVENTOS DA QUADRA
PÓLIESPORTIVA E ATLÉTICAS**

Bryan Carnavale Alvarenga

Kauã Gustavo da Fonseca Rodrigues

Rafael Dias Garcia

Vitor Yago Ferreira dos Santos

Tutor: Prof.º Me. Marco Antônio Gusmão Carvalho

**OURINHOS-SP
2025**

SUMÁRIO

Sumário

1 - INTRODUÇÃO	1
2 - REVISÃO DA LITERATURA.....	3
3 - MATERIAIS E MÉTODOS	4
4 - RESULTADOS E DISCUSSÕES / CONSIDERAÇÕES FINAIS	6
5 - REFERÊNCIAS	9

INTRODUÇÃO

O presente trabalho, desenvolvido pelos alunos do 2º termo B do curso de Engenharia de Software, descreve o projeto e a implementação do Sistema de Gerenciamento de Eventos (SGE). A proposta central foi criar uma plataforma web robusta e centralizada para a organização, agendamento e acompanhamento de eventos no ambiente acadêmico, com foco especial na gestão de atividades de atléticas universitárias.

A situação analisada parte da dificuldade encontrada por estudantes e administradores na gestão manual ou descentralizada de eventos, o que frequentemente resulta em conflitos de agenda, falta de comunicação e dificuldade no controle de presença. O SGE surge como uma solução tecnológica para otimizar esses processos, oferecendo uma interface intuitiva para diferentes perfis de usuários, desde alunos até administradores do sistema.

Os objetivos deste projeto são:

- **Geral:** Desenvolver um sistema web funcional para o gerenciamento de eventos acadêmicos e de atléticas.
- **Específicos:**
 - Implementar um sistema de autenticação com múltiplos níveis de permissão (Super Admin, Admin de Atlética, Professor, Aluno).
 - Fornecer funcionalidades para agendamento, consulta e participação em eventos.
 - Estruturar um banco de dados para armazenar de forma segura as informações de usuários e eventos.
 - Garantir um ambiente de desenvolvimento e implantação padronizado e de fácil reprodução.

O método de trabalho adotado pelo grupo baseou-se em práticas modernas de desenvolvimento de software. Para agilizar a configuração do ambiente de desenvolvimento, partimos de um template de Docker disponibilizado pelo tutor do projeto. A utilização da tecnologia de contêineres garantiu que todos os membros da equipe trabalhassem em um ambiente consistente, simplificando a instalação, os testes e a futura implantação do sistema.

REVISÃO DA LITERATURA

A concepção do Sistema de Gerenciamento de Eventos (SGE) fundamenta-se em conceitos consolidados da engenharia de software e da tecnologia web. A utilização de sistemas de gerenciamento é amplamente documentada como uma ferramenta essencial para a otimização de processos em organizações de todos os portes, incluindo instituições de ensino. Autores da área de sistemas de informação destacam que a centralização de dados e a automação de tarefas, como o agendamento de eventos e a gestão de participantes, reduzem a carga de trabalho manual e minimizam a ocorrência de erros.

O desenvolvimento de aplicações web com a linguagem PHP, apoiada por um sistema de gerenciamento de banco de dados relacional como o MySQL, representa uma das arquiteturas mais testadas e difundidas na indústria. A literatura técnica aponta essa combinação como uma solução de baixo custo, alta performance e com uma vasta comunidade de desenvolvedores, o que facilita a busca por soluções e a manutenção do código.

Adicionalmente, o projeto incorpora a metodologia DevOps por meio do uso de Docker e Docker Compose. A containerização é uma abordagem moderna que, segundo especialistas, resolve o clássico problema do "funciona na minha máquina". Ao empacotar a aplicação e suas dependências em contêineres isolados, garante-se a portabilidade e a consistência entre os ambientes de desenvolvimento, teste e produção, interpretando e aplicando na prática os princípios de integração e entrega contínua.

MATERIAIS E MÉTODOS

Para o desenvolvimento do SGE, o grupo selecionou como dados primários os requisitos funcionais e não funcionais que definem a operação do sistema. Isso inclui as regras de negócio para criação de eventos, os diferentes níveis de acesso e as permissões de cada perfil de usuário (Super Admin, Aluno, Professor, etc.).

A análise desses dados foi realizada a partir de três perspectivas:

- **Instrumental teórico-referencial das disciplinas:** Foram aplicados conceitos de modelagem de banco de dados, arquitetura de software e programação orientada a objetos, estudados ao longo do curso de Engenharia de Software.
- **Informações e conhecimentos construídos pelo grupo:** A equipe definiu a estrutura da aplicação, o esquema do banco de dados e as tecnologias a serem utilizadas (PHP, MySQL, Docker) com base em discussões técnicas e na prototipação das funcionalidades.
- **Experiência e vivência dos estudantes:** A definição dos perfis de usuário e de suas respectivas permissões foi baseada na compreensão das necessidades reais da comunidade acadêmica, visando criar uma ferramenta útil e alinhada às expectativas dos futuros usuários.

Os **Procedimentos Metodológicos** seguiram uma abordagem técnica estruturada:

1. **Controle de Versão:** O código-fonte foi gerenciado utilizando o sistema Git, com o repositório hospedado na plataforma GitHub.
2. **Ambiente de Desenvolvimento:** Foi utilizado o Docker e o Docker Compose para criar e orquestrar os contêineres da aplicação. A estrutura do ambiente foi baseada no template **php-mysql-docker-template**, desenvolvido e distribuído pelo Prof. Me. Marco Antônio Gusmão Carvalho, que inclui contêineres pré-configurados para PHP/Apache, MySQL e phpMyAdmin.

3. **Arquitetura do Software:** A aplicação foi desenvolvida seguindo o padrão de arquitetura **Model-View-Controller (MVC)**. Essa escolha metodológica permitiu a separação clara das responsabilidades do sistema: o **Model** gerencia os dados e a lógica de negócio, a **View** é responsável pela apresentação da interface ao usuário, e o **Controller** atua como intermediário, processando as requisições do usuário e coordenando as interações entre o Model e a View. Isso resultou em um código mais organizado, modular e de fácil manutenção.
4. **Estruturação do Banco de Dados:** O banco de dados, nomeado application, foi modelado para armazenar informações de usuários, eventos, atléticas e permissões. Um script SQL (*db_populate.sql*) foi criado para popular o banco com dados de exemplo, facilitando os testes.
5. **Instalação e Execução:** O processo de instalação foi automatizado por meio de um script entryptoint.sh no contêiner PHP, que instala as dependências do projeto com o Composer. A inicialização completa do ambiente é feita com um único comando: *docker-compose up -d*.

RESULTADOS E DISCUSSÕES / CONSIDERAÇÕES FINAIS

Resultados

Ao final deste projeto integrador, o grupo obteve como resultado principal o desenvolvimento e a entrega de uma aplicação web plenamente funcional, o **Sistema de Gerenciamento de Eventos (SGE)**. Os resultados concretos e tangíveis do projeto são:

- **Plataforma Web Funcional:** Uma aplicação acessível via navegador em `http://localhost`, que permite a interação de diferentes perfis de usuários por meio de um sistema de login seguro e permissões hierárquicas.
- **Ambiente de Desenvolvimento Padronizado:** Um ambiente de desenvolvimento e execução completo e encapsulado com Docker e Docker Compose. A infraestrutura, composta por contêineres para a aplicação PHP/Apache, o banco de dados MySQL e a interface de gerenciamento phpMyAdmin, é iniciada com um único comando (*`docker-compose up -d`*), garantindo a reprodutibilidade e eliminando conflitos de configuração entre as máquinas dos desenvolvedores.
- **Banco de Dados Estruturado:** Um banco de dados relacional, nomeado `application`, foi modelado e implementado para armazenar de forma persistente e organizada todas as informações cruciais do sistema, incluindo dados de usuários, eventos, atléticas e seus respectivos relacionamentos. Um script de povoamento (*`db_populate.sql`*) foi criado para carregar dados de exemplo, viabilizando testes imediatos.
- **Implementação de Múltiplos Perfis:** O sistema contempla cinco perfis de usuário distintos (Super Admin, Admin de Atlética, Professor, Membro de Atlética e Aluno), cada um com um painel de controle e funcionalidades específicas, refletindo as diferentes necessidades da comunidade acadêmica.

Discussões

A construção do SGE permitiu ao grupo não apenas alcançar os objetivos técnicos propostos, mas também gerar discussões e aprendizados significativos. A escolha da arquitetura **Model-View-Controller (MVC)** foi fundamental para o sucesso do projeto. Essa abordagem metodológica impôs uma separação clara entre a lógica de negócios (Model), a interface do usuário (View) e o controle das requisições (Controller). Na prática, isso se traduziu em um código-fonte mais limpo, modular e de fácil manutenção, além de ter facilitado o trabalho colaborativo, permitindo que os membros da equipe atuassem em diferentes partes do sistema simultaneamente sem grandes conflitos.

A utilização do **Docker**, a partir do template fornecido pelo tutor, provou ser outra decisão estratégica acertada. A tecnologia de contêineres resolveu um dos maiores desafios em projetos de software em equipe: a consistência do ambiente. Todos os membros puderam executar uma versão idêntica da aplicação e de suas dependências, o que agilizou o processo de desenvolvimento e depuração. Discutiu-se no grupo que, sem essa ferramenta, muito tempo teria sido gasto com a configuração manual de servidores web e bancos de dados, tempo que foi melhor aproveitado no desenvolvimento das funcionalidades centrais do SGE.

O sistema desenvolvido ataca diretamente o problema levantado na introdução: a falta de uma ferramenta centralizada para a gestão de eventos acadêmicos. Ao oferecer um calendário unificado e funcionalidades de agendamento e inscrição, o SGE mitiga os riscos de conflitos de datas e melhora a comunicação entre organizadores e participantes, representando uma evolução significativa em relação a processos manuais ou descentralizados.

Considerações Finais

O projeto do Sistema de Gerenciamento de Eventos cumpriu com sucesso todos os objetivos estabelecidos. Foi entregue uma solução de software funcional que atende a uma necessidade real do ambiente universitário. Mais do que isso, o projeto serviu como um campo de provas essencial para a aplicação prática dos conhecimentos teóricos adquiridos no curso de Engenharia de Software, solidificando conceitos de arquitetura de software, modelagem de dados e metodologias de desenvolvimento.

Apesar do êxito, reconhecemos que o SGE é um protótipo com espaço para evoluções.

Como **limitações e trabalhos futuros**, identificamos as seguintes possibilidades:

- **Implantação em Nuvem:** Migrar a aplicação de um ambiente local para um servidor em nuvem (como AWS, Google Cloud ou Azure) para torná-la acessível a todos os alunos da instituição.
- **Desenvolvimento de Novas Funcionalidades:** Implementar um sistema de notificações SMS, gerar relatórios por atlética, criar certificados de presença e integrar o sistema com calendários externos (Google Calendar, por exemplo).
- **Melhoria da Interface (UI/UX):** Realizar estudos de usabilidade com usuários reais para refinar a interface, tornando-a ainda mais intuitiva e agradável.
- **Criação de uma Versão Mobile:** Desenvolver um aplicativo móvel ou uma Progressive Web App (PWA) para facilitar o acesso e a interação por meio de smartphones.

Conclui-se que a experiência de desenvolver o SGE foi extremamente enriquecedora, proporcionando ao grupo uma visão completa do ciclo de vida de um produto de software, desde a concepção e planejamento até a implementação e entrega, representando um marco importante na nossa formação acadêmica e profissional.

REFERÊNCIAS

ALVARENGA, Bryan Carnavale; RODRIGUES, Kauã Gustavo da Fonseca; GARCIA, Rafael Dias; SANTOS, Vitor Yago Ferreira dos. **SGE: Sistema de Gerenciamento de Eventos**. 2025. Repositório de Software. Disponível em: <https://github.com/rafaeldiasgarcia/sge>. Acesso em: 24 set. 2025.

CARVALHO, M. A. G. **php-mysql-docker-template**. 2024. Repositório de Software. Disponível em: <https://github.com/marcoweb/php-mysql-docker-template>. Acesso em: 24 set. 2025.

DOCKER. **Docker Documentation**. Disponível em: <https://docs.docker.com/>. Acesso em: 24 set. 2025.

DOCKER. **Docker Compose Overview**. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 24 set. 2025.

GIT SCM. **Git Documentation**. Disponível em: <https://git-scm.com/doc>. Acesso em: 24 set. 2025.

MOZILLA DEVELOPER NETWORK (MDN). **CSS: Cascading Style Sheets**. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/CSS>. Acesso em: 24 set. 2025.

MOZILLA DEVELOPER NETWORK (MDN). **HTML: HyperText Markup Language**. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTML>. Acesso em: 24 set. 2025.

MOZILLA DEVELOPER NETWORK (MDN). **JavaScript**. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. Acesso em: 24 set. 2025.

MOZILLA DEVELOPER NETWORK (MDN). **MVC - Model-View-Controller.**
Disponível em: <https://developer.mozilla.org/en-US/docs/Glossary/MVC>. Acesso em:
24 set. 2025.

MYSQL. **MySQL Documentation.** Disponível em: <https://dev.mysql.com/doc/>.
Acesso em: 24 set. 2025.

PHP. **PHP Manual.** Disponível em: <https://www.php.net/manual/en/>. Acesso em: 24
set. 2025.

PHPMYADMIN. **phpMyAdmin Documentation.** Disponível
em: <https://docs.phpmyadmin.net/en/latest/>. Acesso em: 24 set. 2025.